$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the maximum weighted completion time in a two-machine flow shop models lines on which the most costly order paces the shop. We establish that the problem is strongly NP-hard and give a mixed integer programming formulation for it. We then show that an optimal schedule preserves the permutation property and has a block structure of one block per distinct weight, Johnson's order within each block, and the blocks in non-increasing weight order. On this structure we build three methods. An exact dynamic program prunes dominated states and runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V is the total processing time. A 2-approximation runs in $O(n \log n)$ time, and rounding the time coordinates and weights of the dynamic program turns it into a fully polynomial-time approximation scheme that attains a $(1+\varepsilon)$-approximation for a constant number of weights. For large instances we develop two heuristics, an adapted NEH heuristic and a genetic algorithm. The computational study compares the exact methods on small instances and the heuristics on the large ones, measured against a theoretical lower bound. On the small instances the dynamic program returned the same values as the model and was faster by up to two orders of magnitude. Among the heuristics the genetic algorithm returned the smallest gap in every setting, ahead of the NEH heuristic.

Keywords: Flow shop; Dynamic programming; Approximation algorithms;

1 Introduction

The two-machine flow shop is one of the best known models in scheduling. Its structure is that of a plant whose jobs are machined on one machine and finished on another. Metal

*Corresponding author. E-mail address: author@email.com

parts are cut and then coated, and a casting is rolled after it is poured. A web is printed and then bound, and a granulate is compressed after it is mixed. A job must leave the first stage before it can enter the second, and each stage carries one job at a time. Johnson settled the sequencing decision for this model as early as 1954 (Johnson, 1954). The two-stage structure has remained a standard subject of sequencing theory, and its variants and solution methods are surveyed in (Pinedo, 2022). The criterion of this paper is the maximum weighted completion time, also called the weighted makespan. It charges each job the product of its weight and its completion time and takes the largest of those products. Where the makespan asks when the shop falls idle, the weighted makespan asks which job is left waiting and what that wait costs. The weight belongs to the order rather than to the plant as a whole, and in a plant it is rarely a continuum. A contractual penalty for lateness, the capital tied up in work in process and the priority a large customer has bought are three classes rather than a spectrum. The most costly order is the one carrying the largest of them.

Much the same structure has appeared outside the factory. A system that serves large language model agents handles every step of a task in two phases. The step is reasoned about in the chain of thought (Wei et al., 2022), and only then acted upon, by calling a tool or running code, in the chain of execution (Yao et al., 2023). The two phases draw on different resources, the first on the model that produces the thought and the second on the tool, the interpreter or the index that the call reaches, and the phases of different steps run alongside each other. What settles when a step finishes is therefore the phase it is in and the time the steps ahead of it have kept the resource it is waiting for.

Serving a task takes many such steps, among them a retrieved document, a tool call and a sub-query, and the order in which they are admitted to the two phases settles when the task as a whole finishes. The correspondence with the two-machine flow shop runs through them one by one. A step is a job. The reasoning is the first machine and the acting is the second, and a step cannot be acted upon before it has been reasoned about, exactly as a job cannot enter the second machine before it has left the first. Each phase serves one step at a time, so a step waits for the step that holds the phase ahead of it, exactly as each machine carries one job at a time. The time a step spends in a phase is its processing time on that machine, the latency of the model for the reasoning and of the tool for the acting. The steps are not equal. One bought by a paid tier, or one whose failure is costly, weighs more than the rest, and that cost is the weight of the job.

The environment is not a choice of convenience. The two stages stand in series because a step has to be reasoned about before it is acted upon, and each of them holds one step at a time because the model and the tool each serve one step at a time. A single stage would not do. Reasoning and acting consume different resources, and a step that waits for its tool leaves the model free for the next step, so that merging the two phases would erase the overlap that makes the order of the steps matter at all. The criterion asks for the same environment from the other side. What is judged is the completion time of one step, and on

a single machine that completion time is settled by the position of the step, no earlier stage being there whose delay it could inherit. Two stages in series are what make the completion time of the decisive step a property of the whole schedule rather than of its position in the sequence.

What decides whether the service is acceptable is not the average over the steps but the step that matters most. Weighted by its cost, that step is what the task is judged on, so the criterion is the maximum weighted completion time rather than the average.

In scheduling theory the criterion has been studied in several machine environments. On a single machine, jobs are revealed over time and must be sequenced without knowledge of the future. For this online setting, best-possible algorithms are known for the maximum weighted completion time (Li, 2015). A single machine carrying one restricted rate-modifying activity has also been treated for the same criterion (Zuo et al., 2026). On identical parallel machines, randomized approximation schemes for the weighted makespan have been developed (Sun, 2024). The criterion has also been combined with the decision of which orders to accept (Lu et al., 2025). Maximum-type criteria have been studied in other forms as well. For a single machine, the maximum earliness/tardiness cost has been minimized when jobs may be rejected (Guo et al., 2026). In all of these settings, however, a job occupies one machine for its entire processing time. The interaction between two consecutive stages, which is what defines the flow shop, therefore does not arise.

The flow shop environment, by contrast, has been studied extensively but for different criteria. The variant that has attracted most theoretical attention is the proportionate flow shop, in which every machine spends the same time on a given job. That model, the ordered flow shop and related variants are surveyed in (Panwalkar et al., 2013). Its appeal is that it can often be reduced to a single-machine problem with the same criterion (Panwalkar and Koulamas, 2015), so that results proved for one machine carry over. Weighted objectives have been treated within it as well, but the objective treated is the sum of the weighted completion times rather than their maximum (Shakhlevich et al., 1998). For the general two-machine flow shop, the criterion that has been studied is the makespan, most recently together with the decision of which orders to reject (Shabtay and Gerstl, 2024).

The criterion and the machine environment are therefore each well established, while their combination is not. The criterion has been read on one machine at a time, and the flow shop has been read for the makespan.

The problem is not solved by combining these results, for two reasons. The reductions that make the proportionate flow shop tractable require the processing times to be proportionate. The general two-machine flow shop does not satisfy that assumption, so results proved for one machine do not carry over. More fundamentally, a total weighted completion time decomposes over the jobs. It can therefore be accumulated while a schedule is built, which is what allows a dynamic program to compare partial schedules with one another. A maximum does not decompose in this way, since a partial schedule carries no informa-

tion about its final value until it is complete. Optimizing $F2 \parallel WC_{\max}$ therefore requires structural properties of a kind that the earlier work does not supply.

The criterion matters for the systems above, where a wrong sequence delays the most costly step rather than the average one. It matters for scheduling theory as well, where a maximum weighted criterion has not been placed in the two-machine flow shop.

In this paper we study the problem $F2 \parallel WC_{\max}$. Four features distinguish it from the settings reviewed above. (i) Two machines stand in series, so that each completion time depends on the idle time inherited from its predecessors. (ii) The objective is a maximum over weighted completion times rather than their sum. (iii) The processing times are general, with no proportionate assumption. (iv) No job may be rejected, so that every job must be scheduled. Replacing the weighted sum by a weighted maximum, and dropping the proportionality assumption, would already put the problem beyond the single-machine reduction. We show that it is in fact strongly NP-hard (Theorem 3.2). The difficulty is structural rather than a matter of the instance size. Because the objective is determined by a single job, an exchange that shortens many completion times while lengthening that one does not improve the objective value. Because the jobs carry unequal weights, moreover, Johnson's rule has no direct extension to the weighted case.

Our results follow from the structure of an optimal schedule. Since the objective is a maximum, jobs of equal weight are interchangeable with respect to it. An optimal schedule can therefore be assumed to have a block structure, with one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. On this structure we build three methods, an exact dynamic program, a 2-approximation and an approximation scheme. The dynamic program runs in $O(nKV^{3K-3})$ time, where K is the number of distinct weights and V is the total processing time. It is practical when the weights fall into few classes, as in the plants described above. A 2-approximation runs in $O(n \log n)$ time, and rounding the input of the dynamic program gives a fully polynomial-time approximation scheme. The scheme attains a $(1+\varepsilon)$-approximation for constant K. Because the exact methods remain exponential in K, we also develop two heuristics for large instances and report a computational study that compares them with the 2-approximation.

The rest of the paper is organized as follows. Section 2 fixes the notation and states the completion-time recurrence. Section 3 gives the mixed integer programming formulation and establishes the complexity of the problem. Section 4 proves the structural properties, develops the exact dynamic program, and presents the 2-approximation and the approximation scheme. Section 5 describes the two heuristics. Section 6 reports the computational study, and Section 7 concludes.

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$, the two-machine flow shop with the maximum weighted completion time as the criterion. The name is the three-field notation of scheduling theory (Pinedo, 2022): $F2$ denotes the flow shop in which every job passes through two machines in series, the second field is left empty because the jobs carry no restriction beyond the data below, and $WC_{\max}$ denotes the criterion. In what follows the problem is referred to by this notation alone. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series. Each job J_j must be processed first on M_1 for a_j time units and then on M_2 for b_j time units, and preemption is not allowed. All processing times a_j and b_j are assumed to be positive integers. Each machine can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing sequence on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative processing times on M_1 and M_2 after the first j jobs. The completion time of the job in position j on machine M_2 is

$$C_{\pi(j)} = \max_{1 \le i \le j} \Big\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \Big\}. \tag{1}$$

The objective is to find a schedule π that minimizes the largest of the products $w_{\pi(j)} C_{\pi(j)}$ over the positions,

$$WC_{\max}(\pi) = \max_{1 \le j \le n} w_{\pi(j)}\, C_{\pi(j)}. \tag{2}$$

Here C_j is the completion time of job J_j on M_2 and $C_{\pi(j)}$ is the one of (1). Throughout, $WC^*_{\max}$ denotes the optimal value of $F2 \parallel WC_{\max}$.

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, the positions are indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is assigned to position k, and 0 otherwise;
- C^1_k completion time of the job in position k on M_1;
- C^2_k completion time of the job in position k on M_2;
- C_j completion time of job J_j on M_2;
- Z the maximum weighted completion time $WC_{\max}$;
- M a sufficiently large constant, taken as $2 \sum_{j=1}^{n} (a_j + b_j)$.

The MILP model is:

$$\min \quad Z \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, \ldots, n \tag{5}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, \ldots, n \tag{7}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{9}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, \ldots, n \tag{10}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad j = 1, \ldots, n, \ k = 1, \ldots, n \tag{11}$$

$$Z \geq w_j C_j, \quad j = 1, \ldots, n \tag{12}$$

$$x_{jk} \in \{0, 1\}, \tag{13}$$

$$C_k^1, C_k^2, C_j \geq 0 \tag{14}$$

Constraint (4)–(5) ensure a one-to-one assignment between jobs and positions. Constraints (6)–(7) compute the completion time of the job in each position on M_1. Constraints (8)–(10) compute the completion time of the job in each position on M_2. For $k \geq 2$, the job in position k starts on M_2 only after it has completed M_1 and the preceding job has released M_2. Constraint (11) links the position-based completion times to the job-based completion times through a big-M formulation: when $x_{jk} = 1$, job J_j is in position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$ and the constraint is inactive. Constraint (12) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$ of (2). Constraint (13) requires the assignment variables x_{jk} to be binary. Constraint (14) requires the completion times C_k^1, C_k^2 and C_j to be nonnegative.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given r positive integers $x_1, \ldots, x_r$ with

$\sum_{i=1}^{r} x_i = 2T$, do there exist disjoint subsets $Z_1, Z_2 \subseteq \{1, \ldots, r\}$ with $Z_1 \cup Z_2 = \{1, \ldots, r\}$ and

$$\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T?$$

Construct an instance of $F2 \parallel WC_{\max}$ with $n = r + 1$ jobs. For $j = 1, \ldots, n-1$, set $a_j = 1$, $b_j = rx_j$ and $w_j = T + 1$; for the remaining job J_n, set $a_n = rT$, $b_n = 1$ and $w_n = 2T + 1$. The decision question is whether some schedule satisfies

$$WC_{\max} \leq Y, \qquad \text{where} \qquad Y = r(T+1)(2T+1).$$

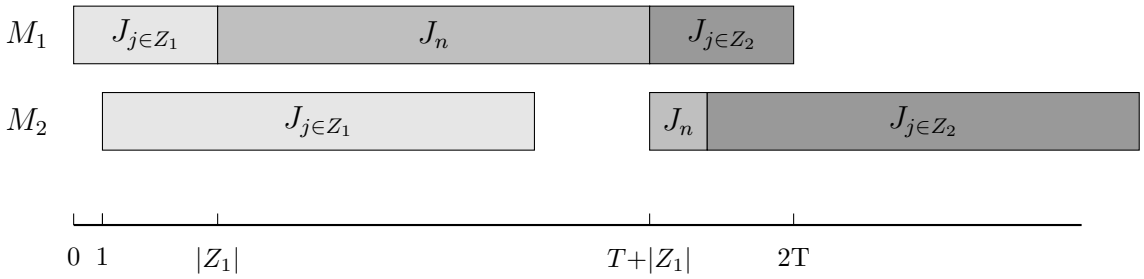


Figure 1

The jobs $J_1, \ldots, J_{n-1}$ are partitioned into two subsets by J_n; the resulting layout on the two machines is shown in Figure 1. The total M_2 processing times of the two subsets are unknown, but their sum equals $2rT$. We may therefore write them as $r(T+\xi)$ and $r(T-\xi)$ for some $-T \leq \xi \leq T$. The completion times of J_n are then

$$C_n^1 = |Z_1| + rT + 1, \tag{15}$$

$$C_n^2 = 1 + r(T+\xi) + 1, \tag{16}$$

and $C_n = \max\{C_n^1, C_n^2\}$.

If $\xi = 0$, then $\sum_{j \in Z_1} b_j = \sum_{j \in Z_2} b_j$, so the Partition instance has a feasible solution. In this case C_n is given by (15), and

$$\begin{aligned} C_n &= |Z_1| + rT + 1 \leq rT + r = r(T+1), \\ w_n C_n &= (2T+1)\, r(T+1) = Y, \\ WC_{\max} &= r(T+1)(C_n + rT) = Y, \end{aligned}$$

so $w_j C_j \leq WC_{\max} = Y$.

Conversely, suppose that some schedule satisfies $WC_{\max} \leq Y$. When $\xi > 0$, C_n is given by (16), and

$$C_n = 1 + r(T+\xi) + 1 \geq 1 + r(T+1) + 1,$$

$$w_n C_n \geq (2T+1)\big(1 + r(T+1) + 1\big) = Y.$$

When $\xi < 0$, C_n is given by (15), and

$$\begin{aligned} C_{\max} &= C_n + \sum_{j \in Z_2} b_j \\ &= 1 + rT + |Z_1| + r(T - \xi) \\ &\geq rT + r + rT \\ &= r(2T+1) = Y, \\ WC_{\max} &\geq r(T+1)\big(r(2T+1)\big) = Y, \end{aligned}$$

so $WC_{\max} > Y$, a contradiction.

The Partition instance therefore admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the strongly NP-complete 3-Partition problem: given $3m$ positive integers $x_1, \ldots, x_{3m}$ and an integer B satisfying $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples, each of sum B?

Let $R = 3m + 5$, $D = RB + 4$, and $T = mD$. We construct an instance with $3m$ element jobs $J_1, \ldots, J_{3m}$ and m separator jobs $S_1, \ldots, S_m$:

- for every element job J_j, set $a_j = 1$, $b_j = Rx_j + 1$;
- for every separator job S_k, set $a_{S_k} = RB + 1$, $b_{S_k} = 1$.

All processing times are positive integers. For $k = 1, \ldots, m$, define $d_k = kD + 1$ and let $Q = (T+2)^3$. Set

$$w_{S_k} = \left\lfloor \frac{Q}{d_k} \right\rfloor, \qquad w_j = \left\lfloor \frac{Q}{T} \right\rfloor \quad (j = 1, \ldots, 3m),$$

and use the decision threshold $Y = Q$. Since $d_k \leq T + 1$ and $Q \geq d_k(d_k + 1)$, we have $\lfloor Q/d_k \rfloor > d_k$. Thus, for every $d \in \{d_1, \ldots, d_m, T\}$ and every integer completion time C,

$$\left\lfloor \frac{Q}{d} \right\rfloor C \leq Q \quad \Longrightarrow \quad C \leq d. \tag{17}$$

Indeed, if $C \geq d + 1$, write $Q = qd + r$ with $q = \lfloor Q/d \rfloor$ and $0 \leq r < d$. Then $q(d+1) = Q - r + q > Q$ because $q > d > r$. Suppose first that the 3-Partition instance is a yes-instance. Let $G_1, \ldots, G_m$ be triples with $\sum_{J_j \in G_k} x_j = B$ for every k, and schedule

$$G_1, S_1, G_2, S_2, \ldots, G_m, S_m.$$

The three element jobs in G_k use 3 units on M_1, and S_k uses $RB+1$ units. Hence S_k completes on M_1 at time kD. The three element jobs in G_k use $RB+3$ units on M_2; since the preceding separator completes on M_2 at time $(k-1)D+1$, the last element of G_k completes at time kD. Therefore

$$C_{S_k} = kD + 1 = d_k.$$

Every element job completes on M_2 no later than $T = mD$. By the definition of the weights, all weighted completion times are at most Q, so $WC_{\max} \le Y$.

Conversely, suppose that a schedule π satisfies $WC_{\max} \le Y$. Since the objective is regular, the standard permutation-schedule property for two-machine flow shops lets us restrict attention to permutation schedules. The separator jobs have identical processing times, so if their positions are fixed, exchanging the labels of two separators that occur in the wrong order can only decrease the maximum weighted completion time, because $w_{S_1} \ge w_{S_2} \ge \cdots \ge w_{S_m}$. We may consequently assume that the separators occur in the order $S_1, S_2, \ldots, S_m$. By (17),

$$C_{S_k} \le kD + 1 \quad (k = 1, \ldots, m), \qquad C_j \le T \quad (j = 1, \ldots, 3m).$$

No element job can occur after S_m. Indeed, if at least one element job did so, consider the last such element job. By the permutation-schedule property invoked above, the processing sequences on M_1 and M_2 coincide, so this job is also the last job on M_1. All m separators and all $3m$ element jobs therefore complete on M_1 no later than it, so its M_1 completion time is at least the total M_1 processing time

$$m(RB+1) + 3m = m(RB+4) = T.$$

Its M_2 completion time would then be strictly greater than T, contradicting $C_j \le T$. The element jobs are therefore partitioned into the m groups G_k lying between S_{k-1} and S_k, where S_0 denotes the start of the schedule. Put

$$B_k = \sum_{J_j \in G_k} x_j, \qquad E_k = \sum_{h=1}^{k} |G_h|, \qquad E_0 = 0.$$

For $k = 2, \ldots, m$, the M_2 path gives

$$C_{S_k} \ge C_{S_{k-1}} + RB_k + |G_k| + 1, \tag{18}$$

while the M_1 path to S_{k-1} gives

$$C_{S_{k-1}} \ge (k-1)(RB+1) + E_{k-1} + 1. \tag{19}$$

Combining (18) and (19) with $C_{S_k} \le kD + 1$ yields

$$R(B_k - B) + E_k \le 3k, \qquad k = 2, \ldots, m. \tag{20}$$

For $k = 1$, the M_2 path and $C_{S_1} \le D + 1$ similarly give

$$R(B_1 - B) + E_1 \le 4. \tag{21}$$

Because $R = 3m + 5$, (20) and (21) imply $B_k \le B$ for every k. Otherwise $B_k \ge B + 1$, and the respective left-hand side exceeds $3k$, or exceeds 4 when $k = 1$. Since every element job belongs to one of the groups, $\sum_{k=1}^{m} B_k = mB$, and hence $B_k = B$ for every k. Finally, $B/4 < x_j < B/2$ implies that a group with sum B contains exactly three elements: at most two elements have sum strictly less than B, while at least four elements have sum strictly greater than B. Thus $G_1, \ldots, G_m$ is a valid 3-partition.

The construction uses $4m$ jobs, and all processing times, weights and the threshold are bounded by a polynomial in m and B. Since 3-Partition remains NP-complete when B is polynomially bounded in m, this is a strong polynomial reduction, and $F2 \parallel WC_{\max}$ is strongly NP-hard. □

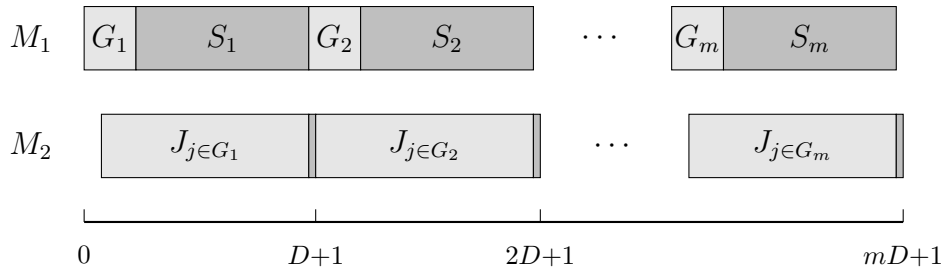


Figure 2: 3-Partition reduction

4 Algorithm

In this section, we design a dynamic programming algorithm for $F2 \parallel WC_{\max}$ and analyze its running time. Throughout, let K denote the number of distinct weights and $V = \sum_{j=1}^{n}(a_j + b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is thus fixed once we decide how each weight class is distributed among the blocks. This is what makes a dynamic program possible, and why the running time of every algorithm below is governed by K.

We build three algorithms on this structure. The first algorithm is exact, and it prunes dominated states. A state records each block's processing time on both machines, the completion time of its last job on M_2, and that job's weight. It runs in $O(n \cdot K \cdot |\mathscr{H}|_{\max})$ time, with $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$

time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1+\epsilon)$-approximation scheme for a fixed number of distinct weights.

4.1 The number of weights is constant

For $F2 \parallel WC_{\max}$, an optimal schedule exists in which both machines process the jobs in the same order (Baker, 1974); we therefore restrict attention to such permutation schedules. Within this class, Johnson's rule (Johnson, 1954) minimizes the makespan: the jobs with $a_j \le b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Fix a permutation schedule π. Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and let

$$\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \qquad k = 1, \ldots, K,$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that is, among $\pi(\ell_{k-1}+1), \ldots, \pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}, \qquad k = 1, \ldots, K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1), \ldots, \pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1, \ldots, \mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1), \ldots, \pi(n)\}, \qquad k = 1, \ldots, K.$$

Hence $\ell_1 \le \ell_2 \le \cdots \le \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2, respectively, C_i the completion time of its last job on M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k, whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \cdots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. *Consider a six-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, *and* $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, *and the resulting blocks are* $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, *and* $\mathcal{B}_3 = \{J_6\}$.

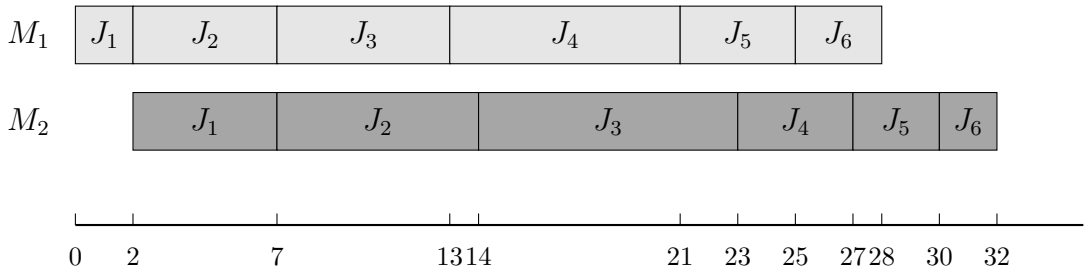


Figure 3

Lemma 4.1. *For any schedule* π, *rescheduling the jobs within each block according to Johnson's rule (Johnson, 1954) and concatenating the blocks in their original order yields a schedule* π' *with* $WC_{\max}(\pi') \leq WC_{\max}(\pi)$.

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k, and every job in $\mathcal{B}_k$ has weight at most L_k: a weight occurring in $\mathcal{B}_k$ either equals W_k, or it has already appeared in an earlier block, whose weights are larger. Let $C_{\pi(\ell_k)}$ denote the completion time of the last job of $\mathcal{B}_k$ in π, the job that sits at position ℓ_k. An empty block satisfies $L_k C_{\pi(\ell_k)} = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j in π, so $C_j \leq C_{\pi(\ell_k)}$, while $w_j \leq L_k$ by the block construction. Hence $w_j C_j \leq L_k C_{\pi(\ell_k)}$, and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j = L_k C_{\pi(\ell_k)}.$$

The objective of (2) therefore equals

$$WC_{\max}(\pi) = \max_{k=1,\ldots,K} L_k C_{\pi(\ell_k)},$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_{\pi'(\ell_k)} \leq C_{\pi(\ell_k)}$ for every k by induction on k.

Base case $k = 1$**.** Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_{\pi'(\ell_1)} \leq C_{\pi(\ell_1)}$.

Inductive step. Assume $C_{\pi'(\ell_{k-1})} \leq C_{\pi(\ell_{k-1})}$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$

begins at the same instant on M_1 under both schedules. On M_2, block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\, C_{\pi(\ell_{k-1})}\}$; the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_{\pi'(\ell_k)} \le C_{\pi(\ell_k)}$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_{\pi'(\ell_k)} \le \max_k L_k C_{\pi(\ell_k)} = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes the jobs sequentially in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \ldots, n$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{\, i : w_j \le W_i \,\},$$

the eligible set is $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. Each state is therefore the aggregate K-block tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

which is described by $4K$ parameters.

The initial state set contains a single state whose components are all zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i, and $C_i \le C'_i$ for every i, then $\mathscr{H}'$ can be discarded without affecting the optimal solution.*

Proof. Consider any continuation that extends $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, and C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$. Since A_i, B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathscr{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i-th slice of the state is updated while all other blocks remain unchanged:

$$A'_i = A_i + a_j, \quad B'_i = B_i + b_j, \quad C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}. \tag{22}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

To support Step 4, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves $F2 \,\|\, WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$ and $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. For constant K, this is $O(nV^{3K-3})$.*

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Initialization] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$, set $\mathscr{H}_0 \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$ and $Z^* \leftarrow +\infty$.

Step 2. [Generation] **for** $j = 1$ **to** n **do**

  for *each* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}$ **do**

    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**

      $A'_i \leftarrow A_i + a_j$, $B'_i \leftarrow B_i + b_j$, $L'_i \leftarrow w_j$

      $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$

      Add the state $(A_1, \ldots, A'_i, \ldots, A_K; B_1, \ldots, B'_i, \ldots, B_K; C_1, \ldots, C'_i, \ldots, C_K; L_1, \ldots, L'_i, \ldots, L_K)$ to $\mathscr{H}_j$

    end

  end

end

[Elimination] Elimination $\mathscr{H}_j$ for $j \in \{1, \ldots, n\}$ by Lemma 4.2

Step 3. [Calculate] **for** *each state* $S = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_n$ **do**

  if $L_i \in \{0, W_i\}$ *for every* $i = 1, \ldots, K$ **then**

    $WC_{\max} \leftarrow$ Algorithm 1 applied to S

    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$

  else

    Discard S.

  end

end

Step 4. [Result] Recover a schedule attaining Z^* by standard backtracking.

Output: An optimal schedule π^*.

Proof. Each state in $\mathscr{H}_j$ is the $4K$-tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\ V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence, $\sum_{i=1}^{K} A_i$ equals the sum of the a_h over that prefix. At most $K - 1$ of the A_i are therefore independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$, the C-vector contributes at most V^{K-1} distinct

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$.

2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule π.

values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L-vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathscr{H}_j| \le 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \dots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \dots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K, 2^K is constant and the bound becomes $O(n \cdot V^{3K-3})$. □

4.3 A 2-Approximation Algorithm

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \le 2\, WC^*_{\max}$.*

Proof. Let π be the schedule returned by Algorithm 3. Its jobs appear in non-increasing order of weight, so we name them $J_1, J_2, \dots, J_n$, with $w_1 \ge w_2 \ge \cdots \ge w_n$ and $\pi = (J_1, J_2, \dots, J_n)$. Every job of the prefix $S = \{J_1, J_2, \dots, J_j\}$ then carries a weight of at least w_j.

We first bound the completion times in π. By (1), the job J_j, which sits at position j of π, completes on M_2 at time $C_{\pi(j)}$, and every term of that maximum is bounded by the full sum:

$$\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h \le \sum_{h=1}^{j} a_h + \sum_{h=1}^{j} b_h = \sum_{h \in S} (a_h + b_h), \qquad 1 \le i \le j.$$

Hence

$$C_{\pi(j)} \;\le\; \sum_{h \in S} (a_h + b_h). \tag{23}$$

Since $WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)}\, C_{\pi(j)}$ ((2)), and $w_{\pi(j)} = w_j$ for the job at position j, multiplying (23) by $w_j > 0$ and taking the maximum over j gives

$$WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)}\, C_{\pi(j)} \le \max_{1\le j\le n} w_j \sum_{h\in S} \big(a_h + b_h\big). \tag{24}$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix $S = \{J_1, J_2, \ldots, J_j\}$ and let π^* be an optimal schedule. Let J_u be the job of S that finishes last on M_1 in π^*, and J_v the job of S that finishes last on M_2. All jobs of S are processed on M_1 before J_u completes M_1, so $C^*_u \ge \sum_{h\in S} a_h$. In the same way, all M_2 processing of the jobs of S precedes the completion of J_v on M_2, so $C^*_v \ge \sum_{h\in S} b_h$. Since both J_u and J_v lie in S, their weights satisfy $w_u, w_v \ge w_j$. The value $WC^*_{\max}$ bounds the weighted completion time of every job of π^*, so $WC^*_{\max} \ge w_u C^*_u \ge w_j \sum_{h\in S} a_h$ and $WC^*_{\max} \ge w_v C^*_v \ge w_j \sum_{h\in S} b_h$.

The prefix S was arbitrary, so the largest of these quantities over the prefixes bounds the optimum as well, and a single job bounds it from below on its own. Three lower bounds on the optimum follow,

$$LB_1 = \max_{1\le j\le n} w_j \sum_{h\in S} a_h, \tag{25}$$

$$LB_2 = \max_{1\le j\le n} w_j \sum_{h\in S} b_h, \tag{26}$$

$$LB_3 = \max_{1\le j\le n} w_j\,(a_j + b_j), \tag{27}$$

each of the three holding for a reason of its own. We call them the M_1 threshold LB_1, the M_2 threshold LB_2 and the job threshold LB_3. The first two charge the workload of the prefix to the job of the prefix that leaves its machine last. That job carries a weight of at least w_j, and it completes its machine only after the whole workload of S has been processed there. Hence $WC^*_{\max} \ge w_j \sum_{h\in S} a_h$ and $WC^*_{\max} \ge w_j \sum_{h\in S} b_h$. The job threshold charges one job its own two operations, which run one after the other and neither of which can be left out, so $C^*_j \ge a_j + b_j$. As J_j carries the weight w_j itself, $w_j(a_j + b_j)$ is a lower bound on the optimum as well. Each bound is determined by the instance alone, and their largest is the reference value of the instance,

$$LB = \max\{LB_1, LB_2, LB_3\}, \tag{28}$$

so that $LB \le WC^*_{\max}$.

For the prefix S, the two bounds give $w_j \sum_{h\in S}(a_h+b_h) \le LB_1 + LB_2 \le 2\,LB \le 2\,WC^*_{\max}$, and the claim follows from (24). □

4.4 FPTAS

4.4.1 Rounding the time coordinates

We now convert the exact dynamic program of Algorithm 2 into a fully polynomial-time approximation scheme by rounding the input data (Hall, 1994). One key point of this

technique is the determination of valid lower and upper bounds for the coordinates to be rounded: each time coordinate ranges in a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a single factor θ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (22) preserves $A_i \le V_A$ and $B_i \le V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \le A_i + B_i \le V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \ge 1 \text{ with } \theta^k \le x < \theta^{k+1}, \qquad \text{and} \qquad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \ge 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \le \theta\, y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathscr{H}^\epsilon = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$ is the tuple formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathscr{H}^\epsilon) = \big(\Delta(A_1), \dots, \Delta(A_K);\ \Delta(B_1), \dots, \Delta(B_K);\ \Delta(C_1), \dots, \Delta(C_K);\ L_1, \dots, L_K\big).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ, where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

The Elimination step keeps one representative per label. At iteration j, every state of $\mathscr{H}^\epsilon_{j-1}$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1, \dots, i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1, \dots, L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \le \theta\, \tilde{x}$ in every time coordinate $x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathscr{H}}^\epsilon$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathscr{H}^\epsilon_j$, and the number of states never exceeds the number of labels.

Lemma 4.3. *Let $\mathscr{H}_j = (A_1, \dots, A_K; B_1, \dots, B_K; C_1, \dots, C_K; L_1, \dots, L_K)$, $j = 1, \dots, n$, be the states along an optimal trajectory of Algorithm 2, that is, the states recovered*

Algorithm 4: State-class FPTAS for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ϵ.

Step 1. [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
$\mathscr{H}_0^\epsilon \leftarrow \{(0,\dots,0;\ 0,\dots,0;\ 0,\dots,0;\ 0,\dots,0)\}$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each state* $(A_1,\dots,A_K;B_1,\dots,B_K;C_1,\dots,C_K;L_1,\dots,L_K) \in \mathscr{H}_{j-1}^\epsilon$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$ **do**
      $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
      $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      [Label] Record the label $\ell(\tilde{\mathscr{H}}_j^\epsilon) =$
       $\big(\Delta(\tilde{A}_1),\dots,\Delta(\tilde{A}_K);\ \Delta(\tilde{B}_1),\dots,\Delta(\tilde{B}_K);\ \Delta(\tilde{C}_1),\dots,\Delta(\tilde{C}_K);\ \tilde{L}_1,\dots,\tilde{L}_K\big)$.
    end
    [Elimination] Among the candidate states sharing the same label, keep one of them and delete the others; set $\mathscr{H}_j^\epsilon$ to the kept states.
  end
end
Step 3. [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathscr{H}_n^\epsilon$.
Step 4. [Result] Same as Step 4 of Algorithm 2.

Output: An approximate schedule π.

by backtracking from an optimal final state. For every j, Algorithm 4 keeps a state $\hat{\mathscr{H}}_j^\epsilon = (\hat{A}_1,\dots,\hat{A}_K;\hat{B}_1,\dots,\hat{B}_K;\hat{C}_1,\dots,\hat{C}_K;L_1,\dots,L_K)$ that carries the same block-end weights $L_1,\dots,L_K$ and satisfies

$$\hat{A}_i \le A_i\,\theta^{j-1},\quad \hat{B}_i \le B_i\,\theta^{j-1},\quad \hat{C}_i \le C_i\,\theta^{j-1},\quad (1 \le i \le K). \tag{29}$$

Proof. We argue by induction on j, from 1 to n.

Base case $j = 1$. Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathscr{H}_1^\epsilon$ and $\mathscr{H}_1$ therefore coincide, and (29) holds with equality at $j = 1$.

Inductive step $j \ge 2$. Assume that (29) holds at $j-1$, and let i be the block to which $\mathscr{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \le W_i$, so $w_j \le W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\hat{\mathscr{H}}_{j-1}^\epsilon$ as well. Write A'_i, B'_i and C'_i for the coordinates of $\mathscr{H}_{j-1}$. Applying the exact recurrences to $\hat{\mathscr{H}}_{j-1}^\epsilon$ and using $\theta^{j-2} \ge 1$ gives a candidate $\tilde{\mathscr{H}}_j^\epsilon$ with

$$\begin{aligned}
\tilde{A}_i &= \hat{A}_i + a_j \le \big(A'_i + a_j\big)\theta^{j-2} = A_i\theta^{j-2},\\
\tilde{B}_i &= \hat{B}_i + b_j \le \big(B'_i + b_j\big)\theta^{j-2} = B_i\theta^{j-2},\\
\tilde{C}_i &= \max\big\{\hat{C}_i + b_j,\ \hat{A}_i + a_j + b_j\big\}\\
&\le \max\big\{C'_i + b_j,\ A'_i + a_j + b_j\big\}\theta^{j-2} = C_i\theta^{j-2},
\end{aligned}$$

where A_i, B_i and C_i are the coordinates of $\mathscr{H}_j$. For the remaining blocks $m \neq i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \leq x'_m \theta^{j-2} = x_m \theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \leq x_m \theta^{j-2}$ holds for every block $m = 1, \ldots, K$. The block-end weights $L_1, \ldots, L_K$ of the candidate are those of $\mathscr{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathscr{H}}_j^\epsilon$ carries the label $\ell(\tilde{\mathscr{H}}_j^\epsilon)$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathscr{H}}_j^\epsilon \in \mathscr{H}_j^\epsilon$ with that same label. Since the label records the block-end weights, $\hat{\mathscr{H}}_j^\epsilon$ carries $L_1, \ldots, L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ, so $y_m^x \leq \theta\, \tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1, \ldots, K$, where y_m^x denotes the coordinates of $\hat{\mathscr{H}}_j^\epsilon$. Together with $\tilde{x}_m \leq x_m \theta^{j-2}$, this yields (29) at j and completes the induction.

Every state of $\mathscr{H}_n^\epsilon$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (29): a derived value would accumulate the rounding errors of the remaining blocks. □

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a fully polynomial-time approximation scheme with*

$$WC_{\max} \;\leq\; (1+\epsilon)\, WC^*_{\max}.$$

For fixed K, the algorithm runs in $O\Big(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\Big)$ time.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathscr{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathscr{H}}_n^\epsilon \in \mathscr{H}_n^\epsilon$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathscr{H}}_n^\epsilon$ are within a factor θ^{n-1} of those of $\mathscr{H}_n$, and its block-end weights are those of $\mathscr{H}_n$, because the weights are not rounded. The final state $\mathscr{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathscr{H}}_n^\epsilon$ has block-end weight at most W_i.

Let $\hat{\pi}$ and π^* be the schedules obtained by concatenating the blocks of $\hat{\mathscr{H}}_n^\epsilon$ and $\mathscr{H}_n$, and let $\hat{C}_i$ and C_i denote the completion times of the last job of block $\mathcal{B}_i$ in the two schedules, which are the corresponding coordinates of the two states. By the concatenation recurrence of Algorithm 1, C_i is nondecreasing in every coordinate of the state and positively homogeneous. Hence $\hat{C}_i \leq \theta^{n-1} C_i$ for every i. Since every block of $\hat{\mathscr{H}}_n^\epsilon$ has block-end weight at most W_i, Lemma 4.1 yields

$$WC_{\max}(\hat{\pi}) = \max_{1 \leq i \leq K} W_i\, \hat{C}_i \leq \theta^{n-1} \max_{1 \leq i \leq K} W_i\, C_i = \theta^{n-1}\, WC^*_{\max}.$$

The factor θ^{n-1} is at most $1 + \epsilon$ for $0 < \epsilon < 1$. Because $1 + x \leq e^x$ for every real x,

$$\theta^{n-1} = \Big(1 + \frac{\epsilon}{2n}\Big)^{n-1} \leq e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \le 1+\epsilon$ on $[0,1]$. The function $g(\epsilon) = 1+\epsilon-e^{\epsilon/2}$ satisfies $g(0)=0$ and

$$g'(\epsilon) = 1 - \tfrac{1}{2}e^{\epsilon/2} \ge 1 - \tfrac{1}{2}e^{1/2} > 0 \qquad (0 \le \epsilon \le 1),$$

because $e^{\epsilon/2} \le e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon) \ge 0$ and $e^{\epsilon/2} \le 1+\epsilon$. Consequently $\theta^{n-1} \le 1+\epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\pi}$, has a value of at most $(1+\epsilon)\,WC^*_{\max}$.

We now bound the running time. At each level j, Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A+2)^K(v^B+2)^K(v^C+2)^K(n+1)^K$. The rounded time coordinates contribute $(v^A{+}2)^K(v^B{+}2)^K(v^C{+}2)^K$, which is at most $(v^C{+}2)^{3K}$ since $v^A, v^B \le v^C$, and each of the K block-end weights takes at most $n+1$ values. Each state generates at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathscr{H}^{\epsilon,n}$ costs $O(K)$ per state. The running time is therefore $O\big(nK\,(v^C{+}2)^{3K}(n{+}1)^K\big)$. Since $\ln(1+\epsilon/(2n)) \ge \epsilon/(4n)$ for $0<\epsilon<1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln \theta} \right\rceil = O\!\left(\frac{n \log V}{\epsilon}\right).$$

For constant K, the bound reduces to $O(n^{4K+1}K(\log V)^{3K}/\epsilon^{3K})$, which is polynomial in n, $\log V$ and $1/\epsilon$. □

4.4.2 Weight rounding

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \le c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\theta), \qquad J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1}) \;\; (1 \le r \le q-1), \qquad J_q = [W_{\min}\theta^q, W_{\max}],$$

with $q = \big\lceil \log_\theta(W_{\max}/W_{\min}) \big\rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\,\theta^{\,r_j+1}, \qquad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \le \hat{w}_j \le \theta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\theta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\pi}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C^*_{\max}$. Then*

$$WC_{\max}(\hat{\pi}) \le \theta\, WC^*_{\max}. \tag{30}$$

Proof. By construction $w_j \le \hat{w}_j \le \theta w_j$ for every job J_j. By (2) and the monotonicity of the maximum, every schedule π satisfies

$$WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)} C_{\pi(j)} \le \max_{1\le j\le n} \hat{w}_{\pi(j)} C_{\pi(j)} = \hat{W}C_{\max}(\pi) \le \theta\, WC_{\max}(\pi).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\pi}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\pi}) \le \hat{W}C_{\max}(\hat{\pi}) = \hat{W}C^*_{\max} \le \hat{W}C_{\max}(\pi^*) \le \theta\, WC_{\max}(\pi^*) = \theta\, WC^*_{\max},$$

which is (30). □

Hence the weight rounding incurs a relative error of at most $\theta = 1 + \epsilon/(2n) \le 1 + \epsilon$, exactly as in the completion-time rounding.

5 Heuristic Algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. For large instances, fast heuristics are therefore of practical interest. In this section we present two heuristics for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Puka et al., 2024) and a genetic algorithm (Qiao et al., 2022).

Throughout this section, the objective of a permutation $\pi = (\pi(1), \ldots, \pi(k))$ of k jobs is evaluated by (1) and (2), so that

$$WC_{\max}(\pi) = \max_{1\le t\le k} w_{\pi(t)}\, C_{\pi(t)},$$

which is computed in $O(k)$ time.

5.1 NEH

Constructive heuristics are attractive for $F2 \parallel WC_{\max}$ because they produce a complete schedule in a single pass, without the tuning effort of population-based metaheuristics. The NEH heuristic (Puka et al., 2024) is among the most effective constructive rules for permutation flow shops, and its core idea is to build a schedule by incrementally inserting jobs into a growing partial sequence, each time choosing the position that best serves the objective.

In the maximum weighted completion-time objective, two properties are particularly important. First, jobs with large total processing time $a_j + b_j$ tend to push the whole makespan forward and therefore should appear early so as not to delay other jobs. Second, given a fixed makespan, jobs with larger weight w_j benefit more from occupying earlier positions, because their weighted completion time decreases. These two properties suggest

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j; let $J_1, \ldots, J_n$ be the resulting sequence.

Step 2. [Initialization] $\pi \leftarrow (J_1)$.

Step 3. [Insertion] **for** $k = 2$ **to** n **do**

  $\pi \leftarrow$ the sequence obtained by inserting J_k into π at a position minimizing $WC_{\max}$.

end

Output: Schedule π.

a greedy priority in which large-$a_j + b_j$ jobs are placed first and large-w_j jobs are used as a tie-breaker.

We adapt the NEH framework as follows. The jobs are first sorted by non-increasing $a_j + b_j$, breaking ties by non-increasing w_j, to obtain an initial sequence $J_1, \ldots, J_n$. The schedule starts with the single job J_1. For each $k = 2, \ldots, n$, the next job J_k is inserted into every one of the k possible positions of the current partial schedule. Each candidate sequence is evaluated by (1), and the position that yields the smallest $WC_{\max}$ is retained. The pseudo-code is given in Algorithm 5.

Because the partial schedule already contains $k - 1$ jobs, the cost of evaluating all k insertions at step k is $O(k^2)$ using the recurrence (1); the total running time is therefore $O(n^3)$ (Theorem 5.1). The heuristic also produces a feasible permutation schedule by construction and requires no hyperparameter selection, so it serves as a fast standalone solver for large instances.

Theorem 5.1. *Algorithm 5 runs in $O(n^3)$ time.*

Proof. At step k, the job J_k is inserted into each of the k positions of a partial schedule of $k - 1$ jobs, and each candidate sequence is evaluated in $O(k)$ time by the recurrence above. Hence step k costs $O(k^2)$, and the total running time is $O\big(\sum_{k=1}^{n} k^2\big) = O(n^3)$. □

5.2 Genetic Algorithm

The objective $WC_{\max}$ is the largest of the products $w_j C_j$ over the jobs. It is a bottleneck criterion, since a single *critical* job attains that maximum and fixes the value of a schedule. An ordering can therefore be improved in only two ways, by lowering the completion time of that job or by shortening completion times across the board. A genetic algorithm for $F2 \parallel WC_{\max}$ should not treat all jobs symmetrically. The genetic algorithm is a standard tool for flow-shop scheduling (Qiao et al., 2022). Nothing in it is built around the weight classes as a whole: the population starts from the order that is shortest on the two machines and knows nothing of the weights, and it is the mutation that moves the blocks of one

weight, one block at a time, until the schedule carries the structure of Lemma 4.1. High-weight jobs should occupy early positions, so that their weighted completion stays small, while the makespan should stay low so that no job is delayed unnecessarily. The algorithm follows the classical framework of a genetic algorithm. It is organized in four parts: the encoding and decoding of a schedule, the generation of the initial population, the iteration of the population, and the mutation of the offspring.

5.2.1 Encoding and decoding

A chromosome is a permutation $\pi = (\pi(1), \ldots, \pi(n))$, in which the gene at position t is the job scheduled t-th. Permutation encoding always represents a feasible permutation schedule, because every job occurs exactly once. No repair operator is therefore needed to restore feasibility after crossover or mutation. To decode a chromosome, both machines process the jobs in the order $\pi(1), \ldots, \pi(n)$. The completion time $C_{\pi(t)}$ of the t-th job follows from the closed form (1), which is evaluated in $O(n)$ time using the prefix sums $A_t = \sum_{h=1}^{t} a_{\pi(h)}$ and $B_t = \sum_{h=1}^{t} b_{\pi(h)}$; the objective value is that of (2),

$$WC_{\max}(\pi) = \max_{1 \le t \le n} w_{\pi(t)}\, C_{\pi(t)}.$$

The fitness of a schedule is the criterion itself, $WC_{\max}(\pi)$, and no normalised scale is fixed for it. Figure 4 shows the encoding and the decoding on the six-job instance of Example 4.1.

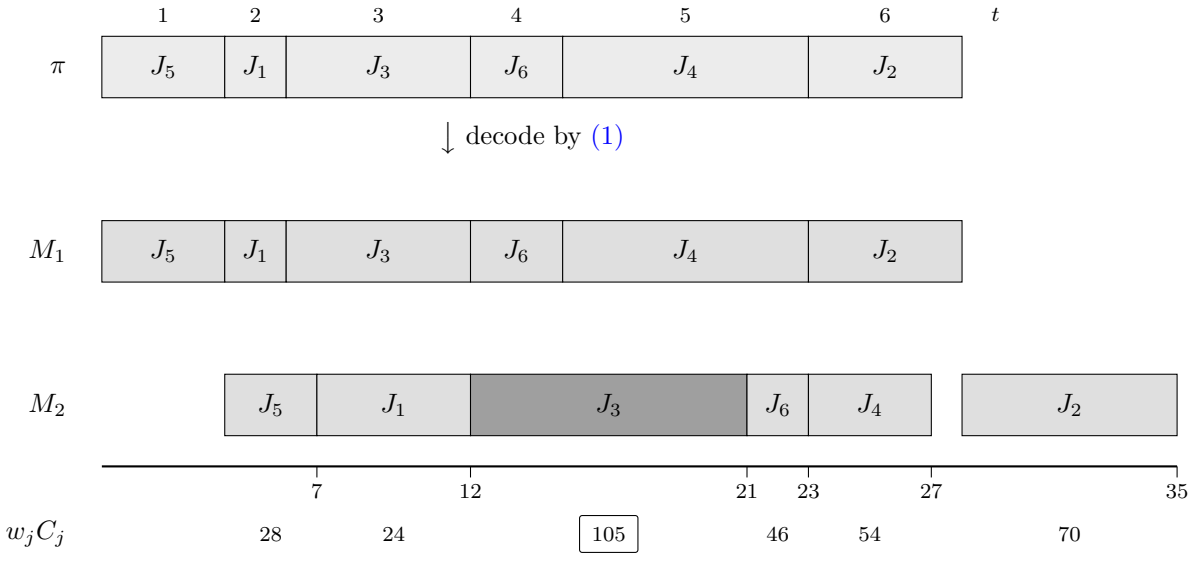


Figure 4: The chromosome $\pi = (J_5, J_1, J_3, J_6, J_4, J_2)$ and the schedule it decodes to, with $w_j C_j$ below each job

5.2.2 Initial population

Random starting populations waste most of the search effort. The initial population P_0 of size N is therefore built from Johnson's rule (Johnson, 1954). Its first individual is the Johnson schedule π^{J} itself, which minimizes the makespan and so shortens every completion time at once. The remaining $N-1$ individuals are neighbours of that schedule, each of them a copy of it with a few neighbouring jobs exchanged, the number of exchanges being drawn uniformly between one and $\lfloor n/4 \rfloor$. A uniformly random permutation carries nothing of the instance, while a neighbour of Johnson's order is already short on the makespan. The rule reads the two processing times alone and knows nothing of the weights, so the population starts from the order that is shortest on the machines and the criterion is brought in by the block move that follows, one weight at a time. The population size grows with the instance and is capped, $N = \min(20, \max(6, \lceil n/2 \rceil))$: a small instance is given a small population, and a large one keeps the same twenty individuals, which are enough for the block mutation and the local search to reach what a population as large as the instance reaches. A generation is dominated by the mutation, which evaluates n positions of a block at $O(n)$ time each, and therefore costs $O(\lambda n^2)$, where λ is the number of offspring the generation produces.

5.2.3 Population iteration

A generation turns the population into its successor by three moves. The parents are selected, the parents are recombined into offspring, and the fittest of parents and offspring survive. The algorithm runs G generations of them, and each of them produces λ offspring from its N parents, so that the operators of one generation are run several times over the same parents. The population it starts from carries nothing of the weights, so the generations are spent building the criterion into the schedule rather than moving an order that already has it, and both G and λ rise with the instance: the depth a run reaches grows with the number of generations, since the elite of one is a parent of the next, and the number of classes an instance has to be sorted into does not. A generation costs the evaluation of the $N + \lambda$ schedules it holds in $O(n)$ time each and one pass of the operators over the offspring, whose most expensive move is the mutation of a child in $O(n^2)$, and the search therefore costs $O(G\lambda n^2)$. What is left of it after the last generation is the local search that closes the algorithm.

Selection. Binary tournament selection forms the mating pool M. In each round, two individuals are drawn uniformly at random from P_{g-1}. The better of the two on the Pareto rank wins and is copied into the mating pool M, ties being broken by the crowding distance. Both individuals are returned to the population, so that a parent may be drawn more than once, and the round is repeated until M_g contains N individuals. Tournament selection is preferred over roulette-wheel selection because it compares two individuals directly rather than through a probability proportional to a fitness value. It therefore remains selective

Algorithm 6: Partially Mapped Crossover (PMX) for $F2 \parallel WC_{\max}$

Input: Parents π_1, π_2; cut points $p_1 < p_2$.

Step 1. [Copy segment] Set $\pi'[t] \leftarrow \pi_1[t]$ for $t = p_1, \ldots, p_2$.
Step 2. [Fill outer positions] **for** $t = 1$ **to** n, $t \notin [p_1, p_2]$ **do**
  $j \leftarrow \pi_2[t]$
  while j *occurs in* π' **do**
    $j \leftarrow \pi_2[\pi_1^{-1}(j)]$ `// follow the PMX mapping`
  end
  $\pi'[t] \leftarrow j$
end

Output: Child π'.

even as the population converges and the schedules of near-optimal individuals become very close.

Crossover. The operator recombines two parent schedules into offspring. These inherit both the relative order and the absolute positions of jobs, the two ingredients a permutation schedule is built from. We use the partially mapped crossover (PMX), the standard operator for permutation-encoded genetic algorithms. Two cut points $p_1 < p_2$ are drawn uniformly at random, and the middle segment $\pi_1[p_1..p_2]$ is copied verbatim into the child. The remaining positions are filled from the second parent π_2. A gene $\pi_2[t]$ of such a position $t \notin [p_1, p_2]$ clashes with the copy only when the copied segment already holds it. The outer positions receive pairwise distinct genes of π_2, so no other clash is possible, and a job cannot be written twice. Such a gene is replaced through the mapping that the segment induces between the two parents, $j \leftarrow \pi_2[\pi_1^{-1}(j)]$. The job π_1 holds at a segment position then gives way to the job π_2 holds at the same position. The replacement is repeated while the gene it yields already occurs in the child. The mapping is injective, so the chain it follows visits distinct jobs and stops after at most $p_2 - p_1 + 1$ replacements. Every position then receives exactly one job, and the child is always a valid permutation. PMX transfers contiguous blocks of jobs, which realize the front-heavy structure identified in Section 4, while still recombining the two parents' orders on the outer positions. Swapping the roles of π_1 and π_2 yields the second child, so that one application replaces the two parents by two offspring. Figure 5 shows both children of one application. Its two upper rows are the parents and its two lower rows the offspring, the grey cells are the middle segment the child copies (π' takes the segment of π_1, π'' that of π_2), and the three double arrows between the parents are the mapping the segment induces. The red cells, numbered in the order in which they are visited, are those of the clash at position 1 of π': the gene taken there, $\pi_2[1] = J_6$, is one the segment already holds, at position 3, and the chain $J_6 \to J_1 \to J_5$ stops at the free job J_5, which is the value the position finally receives.

Survival. The parents of P_{g-1} and the offspring O_g are pooled, so that a good incumbent

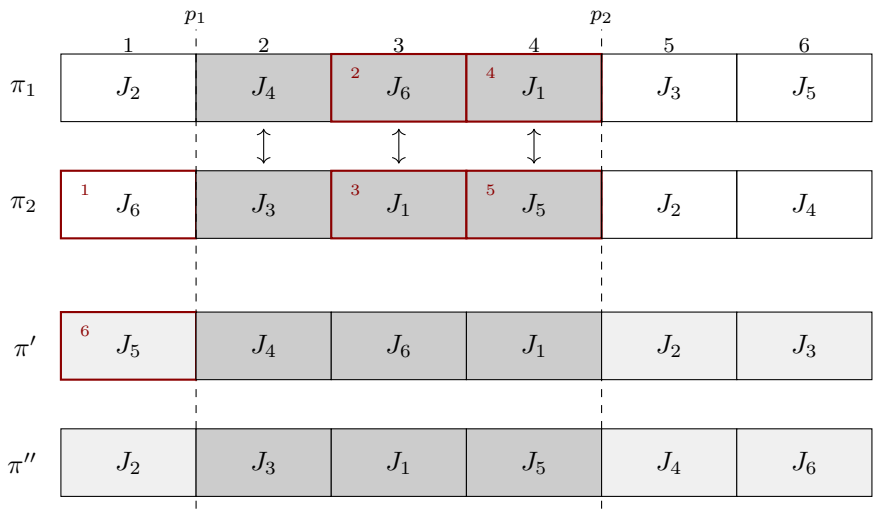


Figure 5: One application of the partially mapped crossover, with $p_1 = 2$ and $p_2 = 4$

is never discarded. The N survivors of $P_{g-1} \cup O_g$ are chosen by the non-dominated sorting and the crowding distance of NSGA-II (Deb et al., 2002). Those rank the pooled individuals on three quantities, the makespan $C_{\max}(\pi)$ of the schedule, the weight $w_{\pi(n)}$ of the job that closes it, and the criterion $WC_{\max}(\pi)$ itself. Whole Pareto fronts are kept while they fit into the budget of N individuals. The front that overflows it is cut by the crowding distance, its least crowded individuals first. The first two carry the criterion: the weighted completion of the closing job is exactly their product,

$$w_{\pi(n)}\, C_{\max}(\pi) \leq WC_{\max}(\pi),$$

so ranking on them keeps both its ingredients in view, a schedule that ends early and one whose last job is light, and keeps the weights in the account, which a search that pressed the makespan alone would leave out. They are not the criterion, however, since a schedule can end early and light and still be a poor one, and a search that ranked them alone would spend its selection on the trade-off between them rather than on what the method is measured on. The criterion is therefore ranked with them rather than tracked beside them. It is also tracked on its own, over every child of every generation, and the best schedule seen on it is carried into the next generation: where the ranking does not keep it, it takes the place of the individual the ranking leaves last, so that an improvement is a parent of the generation that follows and not merely a value that was recorded once.

5.2.4 Mutation

The crossover reads nothing of the weights, so the mutation is what keeps the search moving across the weights: it moves whole weights rather than single jobs, and it is what carries the population across the block orders the criterion rewards.

Algorithm 7: Block Mutation for $F2 \parallel WC_{\max}$

Input: Schedule π; weights w_j.

Step 1. [Block] Evaluate $WC_{\max}(\pi)$ by (1) and let j^* be a job attaining the maximum; with probability 1/2 let $W \leftarrow w_{j^*}$, otherwise let W be a weight drawn at random from the instance.

Step 2. [Move] Form the run of the jobs of weight W, keeping the order π holds them in, and remove it from π.

for *every position r of the remaining schedule* **do**

  Form $\pi^{(r)}$ by inserting the run at position r and evaluate $WC_{\max}(\pi^{(r)})$.

end

$\pi \leftarrow \pi^{(r^*)}$, where $r^* = \arg\min_r WC_{\max}(\pi^{(r)})$, whichever way the criterion moves.

Output: Schedule π.

The move is the one the block structure of Lemma 4.1 suggests. An optimal schedule holds one block per distinct weight, the blocks in non-increasing order of it, and the arrangement the search has to reach is therefore an order of the blocks rather than of the jobs. A single job moved at a time cannot reach it quickly: placing one job leaves the other jobs of its weight where they were, so the search would need as many accepted moves as there are jobs. The mutation therefore moves a block. The jobs of one weight leave the schedule, keeping the order they hold in it, and the run is put back at the position that minimizes $WC_{\max}$ over the whole schedule, which is evaluated by (1) at every one of the positions the run can take. The weight is the one of a job attaining $WC_{\max}(\pi)$, since that is the job that closes the criterion, and with probability 1/2 a weight drawn at random instead, so that the order of the blocks is explored and not only repaired.

The move is a variation and not an improvement. Moving a block forward delays every job it passes, so the criterion may rise while the block order moves towards the one Lemma 4.1 prescribes, and there are orders the criterion rewards that no sequence of accepted moves reaches. The move is therefore proposed whether or not it lowers the criterion, and it is the survival of the generation, and the elite carried through it, that decide what becomes of it.

Genetic Algorithm The four parts above assemble into Algorithm 8, which runs as many generations as the instance is given: the population it starts from is short on the makespan but carries nothing of the weights, so the criterion has to be iterated into the schedule by the mutation before a good order appears. Its closing step is a local search on the schedule the population has reached, over the k jobs that carry the largest weighted completion times, which are reinserted, each at the position that lowers the criterion most, the pass being repeated while it keeps lowering it. Those jobs are the ones that decide the criterion, so reinserting them is where an insertion can still pay. The descent stops as soon as none of them improves.

Algorithm 8: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] Build P_0 from Johnson's schedule and $N-1$ neighbours of it; evaluate all individuals; set π^* to the best.
Step 2. [Evolution] **for** $g = 1$ **to** G **do**
  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.
  Crossover while $|O_g| < \lambda$ **do**
    Draw two parents from M_g; with probability p_c apply Algorithm 6 to obtain two children, otherwise copy the parents into the offspring set O_g.
  end
  Mutation Apply Algorithm 7 to each offspring with probability p_m; evaluate every child by (1).
  Survival $P_g \leftarrow$ the N survivors of $P_{g-1} \cup O_g$, taken by non-dominated sorting on the three objectives and, on the last front, by crowding distance.
  Elite If a child of O_g improves on $WC_{\max}(\pi^*)$, replace π^* by it, and if π^* is not among the survivors, let it take the place of the one the ranking leaves last, so that it is a parent of generation $g+1$.
end
Step 3. [Local search] **repeat**
  Let $J_1, \ldots, J_k$ be the jobs of π^* with the largest $w_j C_j$.
  Form, for every J_i and every position r, the schedule obtained by moving J_i to position r, and evaluate it by (1); let π' be the best of them.
  if $WC_{\max}(\pi') < WC_{\max}(\pi^*)$ **then**
    $\pi^* \leftarrow \pi'$
  end
until *no reinsertion of the step improves*

Output: Best schedule π^*.

6 Experiments

All methods were coded in Python 3.13 and run on a machine with an Intel Core Ultra 5 230F processor and 16 GB of memory, under Windows 11. The MILP model was solved with Gurobi 13.0 on a single thread, so that the reported times are comparable across configurations.

6.1 Experimental setup

Every algorithm was run once on every instance, and no method was handicapped by a smaller budget than another, since all of them evaluate the objective by the same recurrence (1). The dynamic program is read on the number of instances whose optimum it establishes,

since it is the only method whose instance set lies within its reach. Every other method is read on its relative percentage gap against a reference value, reported in per cent, and on the CPU time it requires, reported in milliseconds. Both are given by their mean and by their maximum over the instances of the configuration. The gap of method A is

$$\mathrm{gap}_A = \frac{z_A - z_{\mathrm{base}}}{z_{\mathrm{base}}}, \tag{31}$$

where z_A is the objective value of the schedule it returned and z_{base} is that reference value. On the small scale the reference is the optimum of the instance, which is the value the model returned on it, so that the model itself is read on a gap of zero and every heuristic on its distance from the optimum. On the large scale, where no exact method is within reach, the reference is the lower bound LB of (28): the largest of the three thresholds LB_1, LB_2 and LB_3 of Section 4.3, each of which the instance determines on its own, so that no schedule of a method enters the denominator of its gap.

The parameters of the genetic algorithm were fixed by a calibration run before the study, and each of them is a function of the instance size, so that a single rule covers every size of both scales. Its population is $N = \min(20, \max(6, \lceil n/2 \rceil))$ individuals, which is $6, 8, 10, \ldots, 20$ over the sizes of the small scale and twenty from $n = 40$ on. Both of its probabilities rise with the instance, because the more jobs it holds, the more of the search has to be spent moving them rather than preserving them: the crossover probability is $p_c = \min(1,\ 19/20 + n/4000)$, which rises from 0.95 at $n = 10$ to one at $n = 200$, and the mutation probability is $p_m = \min(1,\ 1/2 + n/500)$, which rises from 0.52 at $n = 10$ to one from $n = 250$ on. The two rates say which operator is expected to do the work. The crossover is applied to almost every pair of parents over the whole study, so it has little room to rise, while the mutation, which carries the population across the weights, is given a share that doubles over it. A small instance is therefore given a small population and mutated sparingly, and a large one the full budget.

Three settings the scales do not share, and each is read with its own pair. The first is the number of generations. The small scale runs $G = \max(2, \lfloor n/4 \rfloor)$ of them, from 2 at $n = 10$ to 15 at $n = 60$. The large one runs $G = \max(16, \lfloor n^2/2000 \rfloor)$, from 16 at $n = 100$ to 80 at $n = 400$: a floor and a quadratic growth, because a generation is one round of the operators and the depth a run reaches is the number of rounds the elite is refined by, which may be neither too small at the bottom of the scale nor growing more slowly than the instance. The second is the number of offspring a generation produces, which the small scale sets to $\lambda = \max(8, \lfloor n^2/12 \rfloor)$ and the large one to $\lambda = \min(200, 60 + \lfloor n/2 \rfloor)$: $8, 18, \ldots, 300$ over the small scale, and 110 at $n = 100$ up to the cap of 200 from $n = 280$ on over the large one. It is what the operators of one generation are run over, so that the generation offers the offspring of its parents λ/N chances rather than the single one a child per parent would give it. The third is the width of the local search of Algorithm 8: the small scale reinserts $k = \max(1, \lfloor n/8 \rfloor)$ jobs, one job in eight of the instance, which is 1, 2 and 7 jobs at

$n = 10$, 20 and 60, and the large one reinserts $k = \max(2, \lfloor 12n/100 \rfloor - 10)$ jobs, twelve jobs in a hundred less ten, which is 2, 14 and 38 jobs at $n = 100$, 200 and 400; the width of the large scale therefore grows faster than the instance itself, one job in fifty of a hundred jobs and one in ten of four hundred. The initial population is built from Johnson's rule on both scales, which reads the two processing times alone, so the criterion has to be built into the schedule by the operators, and the three settings above say how much of the search is spent on that. The two scales are read for different things. The small one is read for the course of the search: its settings are the narrow ones, and the offspring of a generation, rather than the count of them, are what carry it, so that its gap against the optimum falls from 17 per cent at ten jobs to 4 per cent at sixty and closes on the construction rule without passing it. The large one is read for what the search reaches when it is given the room: its settings grow faster than the instance, and its gap closes as the instance grows.

The instances were randomly generated in three groups, and every integer of an instance is drawn from a normal distribution on the integers of its range: the mean is the middle of the range and the standard deviation a sixth of its width, so that the two ends of the range lie three standard deviations from the mean, and the draw is rounded to an integer and clamped to the range. The first group is the one the dynamic program is reported on, and it is the set of the study that preceded this one: the instance sizes 20, 40 and 60, crossed with the weight count $K = 2$, whose weights are the geometric ladder $1, 10$, its exponent drawn by that rule, and with the three processing-time ranges $[1, 20]$, $[1, 40]$ and $[1, 60]$. The dynamic program is not compared with the other algorithms there. Its running time $O(nKV^{3K-3})$ grows so fast in the number K of distinct weights that the sizes the other algorithms are read at are out of its reach, and its instances are therefore not the instances of the comparison; it is reported on a set of its own.

The two groups that follow share their weight setting and their processing-time ranges. An instance carries at most thirty distinct weights, drawn by the rule above on $\{1, \ldots, 30\}$, so that the number of its weight classes is arbitrary below that bound, that the classes near the middle of the range are the populous ones, and that they are neighbouring integers rather than the decades of the first group. Its processing times a_j and b_j are drawn the same way, and independently for every job, on one of the two ranges $[1, 40]$ and $[1, 80]$, which are read as a factor and named in the caption of every table below. The small group varies the instance size over $10, 15, \ldots, 60$ and the large one over $100, 120, \ldots, 400$: the model is compared with the heuristics at the small sizes, where it is exact and settles them, and the heuristics alone are reported at the large ones, where no exact method is within reach and the gap of each of them is measured against the reference value LB of (28). Between them the three groups carry the three factors of the study: the instance size, the number of distinct weights and the width of the processing-time range. Each configuration was replicated over 20 independent instances, and every figure below is the average of those replications. Every method was given the same time limit of 600 s. A method that reaches the limit is recorded with the

Table 1: The dynamic program, $K = 2$. "opt." count optimality.

n	a_j, b_j range	$WC_{\max}$	opt.	time (ms)
20	$[1, 20]$	1243.0	20/20	10.00
		(640.0, 1810.0)		(0.00, 70.00)
	$[1, 40]$	2217.5	20/20	20.00
		(1680.0, 3240.0)		(0.00, 80.00)
	$[1, 60]$	3213.0	20/20	20.00
		(2400.0, 4760.0)		(0.00, 110.00)
40	$[1, 20]$	2225.0	20/20	840.00
		(1530.0, 2720.0)		(110.00, 3260.00)
	$[1, 40]$	4287.0	20/20	4730.00
		(2780.0, 6860.0)		(30.00, 20430.00)
	$[1, 60]$	6929.0	20/20	4810.00
		(4690.0, 9190.0)		(290.00, 19600.00)
60	$[1, 20]$	3155.5	20/20	7930.00
		(2260.0, 4430.0)		(1940.00, 20450.00)
	$[1, 40]$	6480.0	20/20	29670.00
		(4650.0, 8480.0)		(1380.00, 81640.00)
	$[1, 60]$	9402.5	20/20	61250.00
		(6880.0, 12230.0)		(11670.00, 215430.00)

value it has attained rather than omitted. For every configuration the comparison tables give, for each method, the mean and the maximum of its gap and of its running time, each of the two as a pair of figures, and the table of the dynamic program gives the number of instances whose optimum it establishes.

The algorithms of the previous sections were compared on these instances. The model of Section 3 and the two heuristics of Section 5 were applied to every instance of the small group, and the heuristics alone to every instance of the large one, where no exact method is within reach. The dynamic program of Algorithm 2 was applied to the instances of the first group, the only ones its running time reaches, and is reported on its own. What the tables read is the instance size, and, from one table to the next, the width of the processing-time range.

The tables below carry the results, and each is read for one question. Table 1 reports the dynamic program on its own instance set. Tables 2 to 3 report the small scale, one table per processing-time range, and are read for the gap and the running time of every method as the instance size grows. Tables 4 to 5 report the large scale, one table per range, and are read for the gap and the running time of each heuristic under the instance size. The four tables are read together, and what they report is a change of leader rather than a fixed

order: which method comes closest to the reference depends on the scale, and on the size within a scale.

6.2 Results

The model settled every instance of the small scale to optimality – 20 of 20 in each of its 22 configurations, 440 instances in all – so that the gaps reported against it below are distances to an optimum and not to the incumbent of a run that ran out of time. Its cost is what grows. The mean time of an instance rises from 50.6 ms at ten jobs to 80.1 s at sixty on $[1, 40]$ and from 40.2 ms to 43.5 s on $[1, 80]$, and the value it returns rises with the range as well: the mean optimum of a ten-job instance is 2510.7 on $[1, 40]$ and 5433.0 on $[1, 80]$, and that of a sixty-job instance 12323.6 and 25266.1. The slowest instance of the scale takes 299.6 s of the 600 s a run is allowed, so the exact side is complete on this scale but not roomy: the limit is close by the top of it.

The dynamic program is reported at $K = 2$ alone, and what limits it is memory rather than time. Its state space is bounded by $2^K V^{3K-3}$, with V the total processing time of the instance (Section 4), so that a third weight class multiplies the number of states by $2V^3$: at $K = 3$ the memory of the machine the runs were made on is exhausted and the program does not return. What Table 1 reads is the effect of the two parameters that remain. The first is the instance size: at $a_j, b_j \in [1, 40]$ the mean running time rises from 20 ms at twenty jobs to 4.73 s at forty and 29.67 s at sixty, while the value of the criterion grows only from 2217.5 to 4287.0 and 6480.0 over the same three sizes. The second is the width of the processing-time range, which enters the state space through V rather than through the number of jobs: at sixty jobs the mean time is 7.93 s on $[1, 20]$, 29.67 s on $[1, 40]$ and 61.25 s on $[1, 60]$. Every configuration of the group settles all twenty of its instances to optimality, so what the group varies is the cost of the exact method and not the answer it returns.

The two rules that read an instance without searching it are close to the optimum and far apart in what they cost. The 2-approximation, which reads the weights alone, has a mean gap of 3.52 per cent over the whole scale – 3.47 on $[1, 40]$ and 3.56 on $[1, 80]$ – and reaches the optimum on 36 of the 440 instances, its worst instance 16.71 per cent away. NEH has a mean gap of 2.53 per cent, 2.39 on $[1, 40]$ and 2.67 on $[1, 80]$, reaches the optimum on 99 of the 440, and its worst instance is 11.27 per cent away. The two reverse over the scale: NEH is the closer of them at nine of the eleven sizes on $[1, 40]$ and at seven of them on $[1, 80]$, and the 2-approximation takes the lead at the largest sizes, where the order of the weights is what the criterion rewards. Their costs are not of the same order either: 0.02 ms an instance on average for the 2-approximation, and never above 0.05 ms, against 2.68 ms for NEH, whose cost grows with the instance to 8.35 ms at sixty jobs, the model being a thousand times above both at 13.85 s an instance on the average of the scale.

The genetic algorithm is the only method of the three whose gap closes over the scale,

and here it closes onto the other two without passing them. Its mean gap falls from 16.83 to 3.92 per cent on $[1, 40]$ and from 13.16 to 3.24 per cent on $[1, 80]$, which leaves it within a point of both rules at sixty jobs. Its course is not monotone: it rises to 18.96 and 18.97 per cent at fifteen jobs and stays above 14 per cent until thirty-five, where the instances are harder for it than the ten-job ones by more than the growth of its own budget makes up for. Averaged over the 22 configurations it stands at 10.75 per cent, against 3.52 for the 2-approximation and 2.53 for NEH, and its worst instance of the scale is 67.41 per cent from the optimum, at ten jobs. Its cost is above both rules by three orders of magnitude: 0.55 ms an instance at ten jobs and 1.65 s at sixty, against the 80.1 s the model needs at that size.

Table 2: The small scale, $a_j, b_j \in [1, 40]$. An entry of "gap" or "time" is the pair (mean, maximum) of that quantity

n	MILP		2-approx.		NEH		GA	
	$WC_{\max}$	time (ms)	gap	time (ms)	gap	time (ms)	gap	time (ms)
10	2510.7 3192.0	50.61 80.45	4.90 14.18	0.01 0.01	**0.78** 8.68	0.08 0.27	16.83 67.41	0.55 0.75
15	3682.3 4746.0	134.97 275.95	2.73 8.70	0.01 0.02	**2.03** 7.97	0.18 0.22	18.96 42.04	2.48 2.71
20	4742.4 5759.0	377.45 886.36	4.57 9.48	0.01 0.02	**2.29** 9.71	0.39 0.42	14.22 34.19	10.27 11.21
25	5736.1 7260.0	725.39 2338.97	3.75 10.79	0.01 0.02	**1.83** 9.19	0.69 0.73	11.49 27.85	26.07 28.68
30	6513.6 8892.0	1269.43 2396.07	4.37 9.34	0.01 0.03	**2.85** 9.31	1.14 1.20	14.67 43.61	59.60 64.39
35	7836.4 8931.0	2973.48 10776.15	**2.55** 9.68	0.02 0.03	2.67 7.67	1.74 1.79	15.00 30.68	117.27 126.27
40	8760.2 10153.0	5106.22 15912.26	3.08 7.69	0.02 0.03	**2.50** 8.71	2.53 2.61	8.42 21.17	240.93 262.61
45	9564.3 11076.0	12793.08 37697.87	3.63 7.98	0.02 0.03	**2.64** 7.12	3.52 3.66	6.03 17.02	403.04 443.34
50	10631.6 11774.0	19520.52 61667.95	2.65 7.24	0.02 0.03	**2.45** 7.10	4.92 5.37	6.88 21.46	659.74 732.34
55	11760.6 12992.0	36700.15 176554.12	3.02 6.35	0.02 0.05	**2.96** 8.08	6.29 6.50	3.32 10.77	1021.85 1099.12
60	12323.6 14703.0	80106.40 299585.38	**2.93** 8.87	0.02 0.04	3.31 6.72	8.05 8.35	3.92 10.07	1652.00 1721.89

On the large scale, where no exact method is within reach and the reference is the lower bound LB, the three methods separate for good. The 2-approximation is the closest on average, at 1.81 per cent over the 32 configurations, its mean never above 3.20 per cent and its worst instance 7.57 per cent away. NEH is the furthest, at 4.40 per cent, and its gap hardly moves over the scale: 5.02 per cent at a hundred jobs against 3.60 at four hundred on $[1, 40]$, and 5.39 against 4.47 on $[1, 80]$, its worst configuration 5.59 per cent and its worst instance 12.64. The genetic algorithm is the only method whose gap falls as the instance grows, and it falls steeply: 10.88 per cent at a hundred and eighty jobs, 5.49 at two hundred, 2.09 at two hundred and forty and 0.22 at four hundred on $[1, 40]$, and 6.99, 6.34, 2.09 and 0.13 per cent on $[1, 80]$. It passes NEH at 240 jobs and the 2-approximation at about 260 on $[1, 80]$ and 280 on $[1, 40]$, and over the 640 instances of the scale it attains the best value on 318 of them, against 283 for the 2-approximation and 42 for NEH. Its worst configuration

Table 3: The small scale, $a_j, b_j \in [1, 80]$. An entry of "gap" or "time" is the pair (mean, maximum) of that quantity

n	MILP		2-approx.		NEH		GA	
	$WC_{\max}$	time (ms)	gap	time (ms)	gap	time (ms)	gap	time (ms)
10	5433.0 7365.0	40.18 86.55	4.63 16.71	0.01 0.01	**0.62** 2.07	0.08 0.16	13.16 41.92	0.53 0.58
15	6941.7 9035.0	163.01 299.74	4.17 11.66	0.01 0.02	**2.15** 8.18	0.18 0.20	18.97 37.17	2.32 2.69
20	9029.8 10906.0	386.24 653.07	3.84 13.68	0.01 0.02	**1.31** 7.42	0.39 0.49	12.60 37.31	10.02 11.10
25	10971.5 13013.0	692.05 1391.04	3.97 11.14	0.01 0.03	**3.08** 10.71	0.71 0.75	16.01 27.13	26.02 28.33
30	12832.5 15232.0	1869.80 9374.60	3.86 8.81	0.01 0.02	**3.40** 7.90	1.14 1.17	12.93 30.56	57.85 65.72
35	15171.0 16940.0	3264.38 10566.47	3.83 7.84	0.02 0.02	**3.09** 11.27	1.75 1.82	10.00 19.84	115.93 124.35
40	16912.4 19513.0	8283.81 25306.71	**2.94** 11.28	0.02 0.03	3.12 11.03	2.54 2.66	10.37 29.73	237.39 255.75
45	19067.3 23062.0	9174.69 18458.57	2.96 7.32	0.02 0.04	**2.62** 7.82	3.55 4.00	7.10 23.50	405.21 435.56
50	20795.4 23505.0	22314.68 51257.77	3.52 9.63	0.02 0.03	**3.44** 9.71	4.78 4.94	5.54 14.98	657.49 705.72
55	22517.8 27573.0	55263.13 140840.34	**2.77** 9.26	0.02 0.05	3.72 10.07	6.23 6.42	6.76 17.55	1021.42 1151.79
60	25266.1 29614.0	43535.09 103537.34	**2.71** 8.44	0.03 0.04	2.80 6.90	7.97 8.18	3.24 8.86	1652.52 1825.83

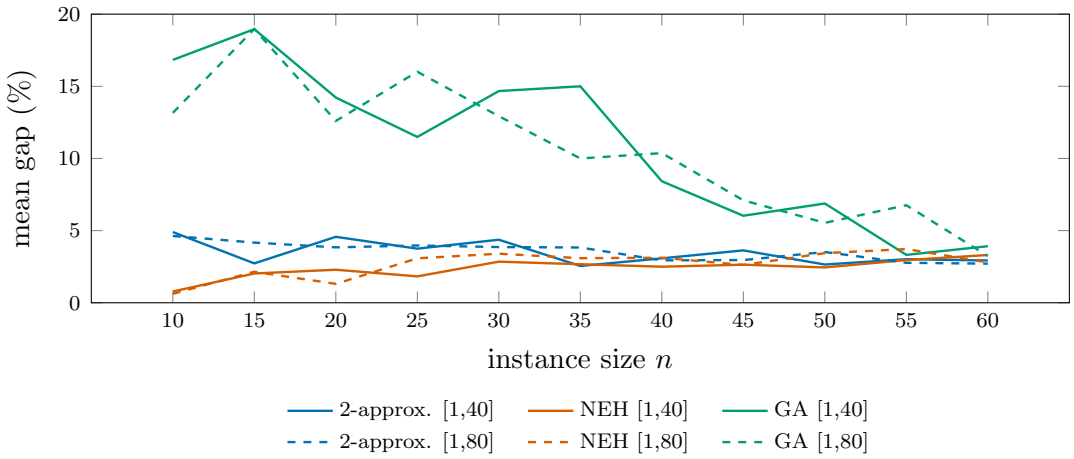


Figure 6: Mean relative gap of the three heuristics on the small scale, against the optimum the model returned. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

and its worst instance are both at the bottom of the scale, at 10.88 and 28.17 per cent. The cost of that is 70.8 s an instance on average, against 0.076 ms for the 2-approximation and 0.76 s for NEH, and 313.5 s on the slowest instance of the scale, of the 600 s a run is allowed.

The two ranges agree on the ordering of the two rules and disagree on the search. The 2-approximation is closer on the wider range, 1.70 per cent against 1.92, and NEH further from it, 4.79 against 4.02, so that the 2-approximation leads NEH on both ranges while the search overtakes it on one: 4.28 against 4.02 per cent on $[1, 40]$, where the two are within a

quarter of a point of one another, and 3.65 against 4.79 on $[1, 80]$.

Table 4: The large scale, $a_j, b_j \in [1, 40]$. An entry of "gap" or "time" is the pair (mean, maximum) of that quantity

n	2-approx.		NEH		GA	
	gap	time (ms)	gap	time (ms)	gap	time (ms)
100	**3.20** 7.57	0.03 0.07	5.02 7.69	35.26 36.47	10.58 28.17	1384.37 1439.61
120	**2.87** 5.08	0.03 0.05	4.26 11.00	61.35 67.38	9.71 21.40	2203.41 2271.48
140	**2.69** 6.12	0.03 0.04	4.08 6.16	92.04 97.28	10.76 23.19	3180.44 3283.81
160	**1.89** 3.70	0.04 0.04	4.59 7.14	134.46 137.52	9.58 20.50	4555.65 4747.43
180	**2.35** 5.34	0.04 0.07	4.11 7.33	191.30 198.87	10.88 20.41	6435.09 6629.00
200	**2.06** 5.11	0.05 0.06	3.89 8.45	260.86 266.55	5.49 14.18	10931.38 11172.88
220	**2.00** 5.12	0.06 0.10	3.90 5.88	344.81 353.78	4.33 10.78	17399.07 17848.49
240	**1.65** 3.54	0.06 0.10	4.21 7.15	445.75 455.58	2.09 8.10	26442.75 27195.43
260	**1.67** 3.88	0.07 0.08	4.02 6.73	568.66 576.57	1.73 5.95	39325.56 40106.05
280	1.63 4.59	0.07 0.08	3.97 7.33	720.96 737.22	**0.81** 3.79	57650.47 58582.96
300	2.05 5.43	0.08 0.11	4.11 6.21	892.01 904.59	**0.68** 1.97	77091.75 79319.03
320	1.35 2.71	0.09 0.14	3.59 6.12	1090.79 1111.77	**0.59** 2.36	99565.36 101622.73
340	1.31 4.02	0.10 0.15	3.54 5.47	1313.39 1325.81	**0.43** 1.94	126205.10 127684.57
360	1.16 2.85	0.11 0.14	3.67 7.22	1562.68 1583.38	**0.19** 0.94	158777.05 160802.00
380	1.31 3.08	0.12 0.15	3.79 5.99	1935.62 2143.48	**0.37** 1.59	205483.27 219431.17
400	1.51 2.99	0.20 0.89	3.60 4.93	2405.22 2877.28	**0.22** 1.06	276234.16 331651.06

The two scales are read for different things, and their tables show why. On the small scale the three methods end within a point of one another, the search having travelled the whole way to join the two rules; on the large one the rules stay within a few points of the bound at every size while the search falls through both, and it is there, and only there, that the population buys a value the rules do not reach.

6.3 Discussion

What the study reads, then, is a division of labour rather than a ranking. The 2-approximation is the cheapest method by six orders of magnitude and stays within five per cent of the optimum on the small scale and within 1.81 per cent of LB on the large one, which says that the factor of two of Theorem 4.2 is loose by an order of magnitude on instances of this kind. What it cannot do is read anything but the weights, and that is the ground NEH takes: NEH reads the two processing times and no weight, which makes it the

Table 5: The large scale, $a_j, b_j \in [1, 80]$. An entry of "gap" or "time" is the pair (mean, maximum) of that quantity

n	2-approx.		NEH		GA	
	gap	time (ms)	gap	time (ms)	gap	time (ms)
100	**2.75** 5.26	0.03 0.05	5.39 12.64	35.66 37.43	8.95 23.55	1409.90 1496.96
120	**2.44** 4.66	0.03 0.06	4.64 6.47	60.32 66.07	9.06 20.88	2167.31 2249.25
140	**2.15** 6.02	0.03 0.04	4.81 8.43	90.89 93.33	7.24 15.10	3136.94 3209.90
160	**2.60** 5.49	0.04 0.04	3.97 7.54	134.96 139.10	10.21 21.92	4571.04 4726.96
180	**2.14** 4.70	0.04 0.05	4.66 7.17	190.18 195.30	6.99 17.08	6429.74 6726.51
200	**1.87** 6.60	0.05 0.07	5.00 8.60	260.31 265.96	6.34 17.48	10916.49 11382.00
220	**1.37** 2.98	0.05 0.06	5.11 7.64	342.95 347.44	2.47 7.33	17326.84 17825.02
240	**1.26** 2.43	0.06 0.07	4.64 8.01	445.17 460.79	2.09 6.77	26299.84 27028.61
260	1.87 4.87	0.07 0.08	5.59 8.02	569.50 582.71	**1.85** 5.73	39464.28 40446.42
280	1.41 3.11	0.07 0.09	3.90 6.46	720.36 731.04	**0.74** 2.42	57760.02 58914.31
300	1.20 2.92	0.09 0.13	4.90 6.68	891.43 907.87	**1.09** 3.79	76949.10 78040.10
320	1.23 3.09	0.09 0.12	4.30 6.03	1092.38 1114.11	**0.51** 1.78	99498.39 100647.08
340	1.05 3.00	0.10 0.12	4.63 6.59	1312.40 1329.73	**0.20** 1.02	125947.63 126830.87
360	1.17 2.17	0.11 0.13	5.37 6.78	1586.33 1648.92	**0.24** 1.43	160224.94 164703.79
380	1.60 3.28	0.12 0.13	5.21 7.25	1935.08 1975.71	**0.36** 1.80	208505.05 212639.22
400	1.11 3.71	0.20 0.43	4.47 7.66	2729.24 3169.58	**0.13** 0.65	313465.36 350424.29

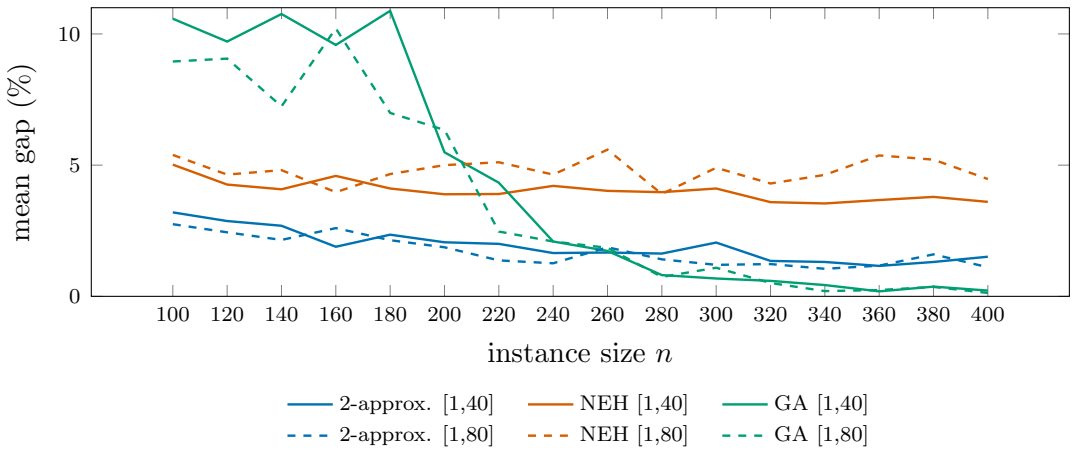


Figure 7: Mean relative gap of the three heuristics on the large scale, against the lower bound LB. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

closest method at the bottom of the small scale and the furthest at the top of the large one, where its gap does not close as the instance grows. The genetic algorithm is the only method whose gap falls with the instance and the only one that beats both rules, from about 260 jobs on and at ninety-three times the running time of NEH. The reason is the start it is given: Johnson's rule carries no weight, so the search spends its early generations building the criterion into the schedule before it can profit from it.

Two properties of the comparison are worth keeping in view. The large-scale gaps are distances to LB, which caps the optimum by the three prefix bounds of (28) and is tight only where the heaviest jobs are also the earliest ones; what they order reliably is therefore the methods and not their absolute quality. And the answers depend on the processing-time range: the average advantage of the search over NEH, 3.97 against 4.40 per cent, is earned on $[1, 80]$ alone.

What would extend the study follows from each of its limits. On the exact side the dynamic program is stopped by memory, so dominance rules that drop a state another state beats, or a solution of one weight class at a time, are the way past $K = 2$, with the scheme of Section 4.4 the memory-lean alternative where they are not enough. On the side of the search, seeding the population with the order the 2-approximation returns costs microseconds and carries the weights the Johnson order lacks, which is the cheap way to move the size at which the search overtakes the rules. Both point at the same economy, of spending the budget where the criterion still moves, which the calibration of the large scale measured when it found the count of generations to be the lever of that scale and the width of the insertion descent nearly inert.

7 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling.* Wiley, New York.

Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Guo, L., Xia, F., and Zhang, X. (2026). Single machine scheduling to minimize maximum earliness/tardiness cost with job rejection. *CCF Transactions on High Performance Computing*, 8(3):314–330.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Li, W. (2015). Best-possible online algorithms for single machine scheduling to minimize the maximum weighted completion time. *Asia-Pacific Journal of Operational Research*, 32(4):1550030.

Lu, L., Zuo, L., Zhang, L., and Ou, J. (2025). Order acceptance and scheduling with weighted makespan. *4OR*, 23(3):247–267.

Panwalkar, S. S. and Koulamas, C. (2015). On equivalence between the proportionate flow shop and single-machine scheduling problems. *Naval Research Logistics*, 62(7):595–603.

Panwalkar, S. S., Smith, M. L., and Koulamas, C. (2013). Review of the ordered and proportionate flow shop scheduling research. *Naval Research Logistics*, 60(1):46–55.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vn-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Shakhlevich, N., Hoogeveen, H., and Pinedo, M. (1998). Minimizing total weighted completion time in a proportionate flow shop. *Journal of Scheduling*, 1(3):157–168.

Sun, R. (2024). Randomized approximation schemes for minimizing the weighted makespan on identical parallel machines. *Journal of Combinatorial Optimization*, 47:34.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Zuo, L., Zhang, J., Shi, Z., and Fan, B. (2026). Single machine scheduling with a restricted rate-modifying activity to minimize the weighted makespan. *Discrete Applied Mathematics*, 380:89–100.